
`github.com/mg-i-roc-maz/corsi · microservizi/lezione-2`

LEZIONE 2 DI 8 · 3 ORE

Docker e Docker Compose

Ogni servizio nel suo container e tutto il negozio, database compreso, avviato con un solo comando.

Le tre ore di oggi

Teoria · 45'

Teoria

Container e immagini, Dockerfile e layer, volumi, porte, reti, variabili d'ambiente, Compose.

Laboratorio · 2h15

Laboratorio

Dockerfile multi-stage, catalog con PostgreSQL in Compose, orders come secondo servizio, debug da VS Code.

Ultimi 10'

Una domanda per chiudere: «cosa si rompe se...?»

DA DOVE RIPARTIAMO

Nella lezione 1 abbiamo staccato catalog dal monolite. Oggi lo mettiamo in un container, con un database vero.

LEZIONE 1

catalog sulla 3001, dati in memoria: si perdono a ogni riavvio.

OGGI

catalog e orders in due container, prodotti su PostgreSQL.

LEZIONE 3

orders chiede a catalog prezzi e scorte via HTTP.

«Sul mio computer funziona»

Lo stesso codice si comporta in modo diverso su macchine diverse. Le cause tipiche:

Versioni diverse

Node 18 su un PC, Node 22 su un altro. PostgreSQL 14 in produzione, 16 in sviluppo.

Pezzi mancanti

Il database non è installato, o è configurato in modo diverso. Manca una libreria di sistema.

Configurazione

Porte, password e indirizzi scritti a mano su ogni macchina, ognuno a modo suo.

Un **container** impacchetta il programma con **tutto** quello che gli serve: stesso comportamento ovunque.

Un container è un processo isolato

- Ha il **suo filesystem**: vede solo i file dell'immagine
- Ha la **sua rete**: indirizzo e porte propri
- Vede solo i **suoi processi**
- Usa il **kernel** del sistema che lo ospita
- Parte in **un secondo**: non avvia un sistema operativo

APPROFONDIMENTO · COME FUNZIONA

Namespace

Decidono cosa vede il processo: file, rete, altri processi, utenti.

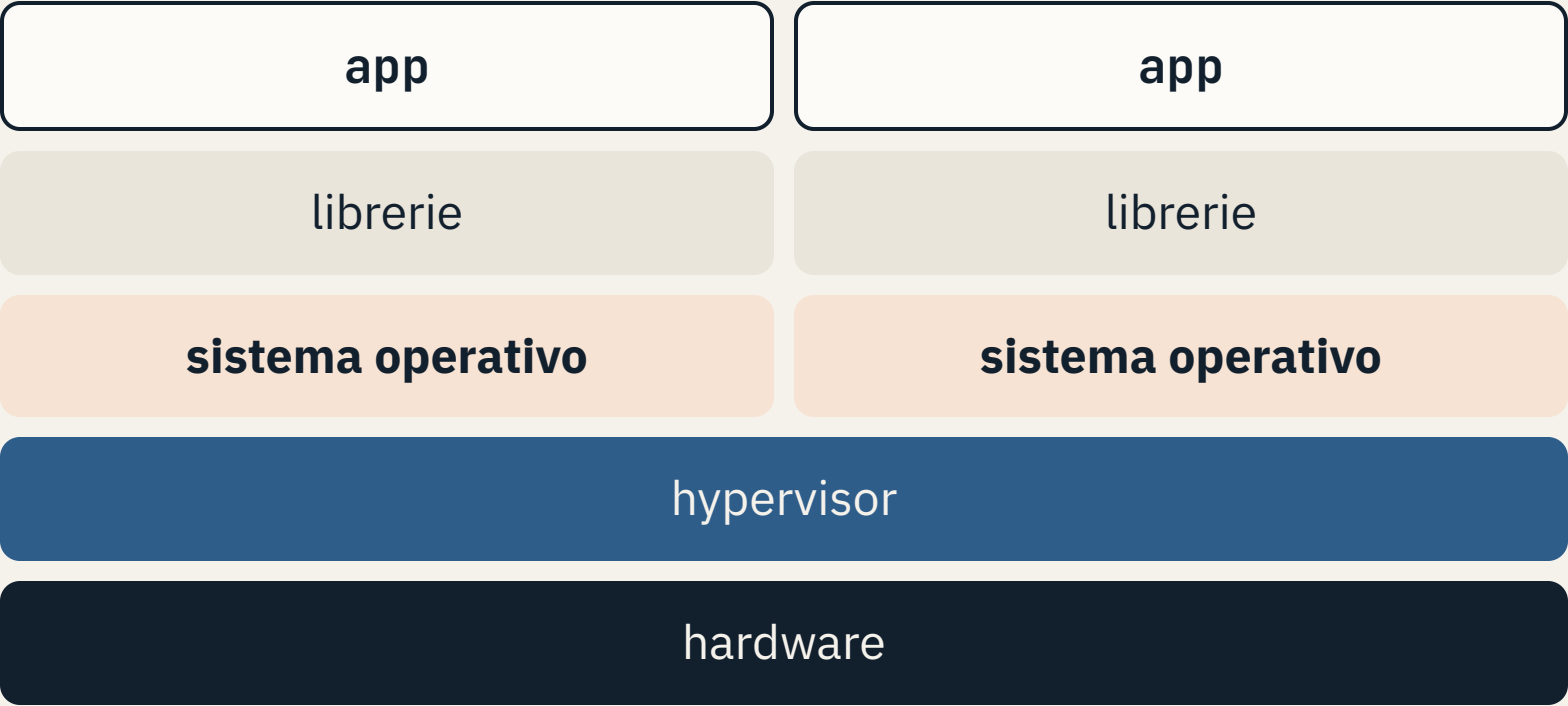
Cgroup

Decidono quanto può usare: CPU, memoria, disco.

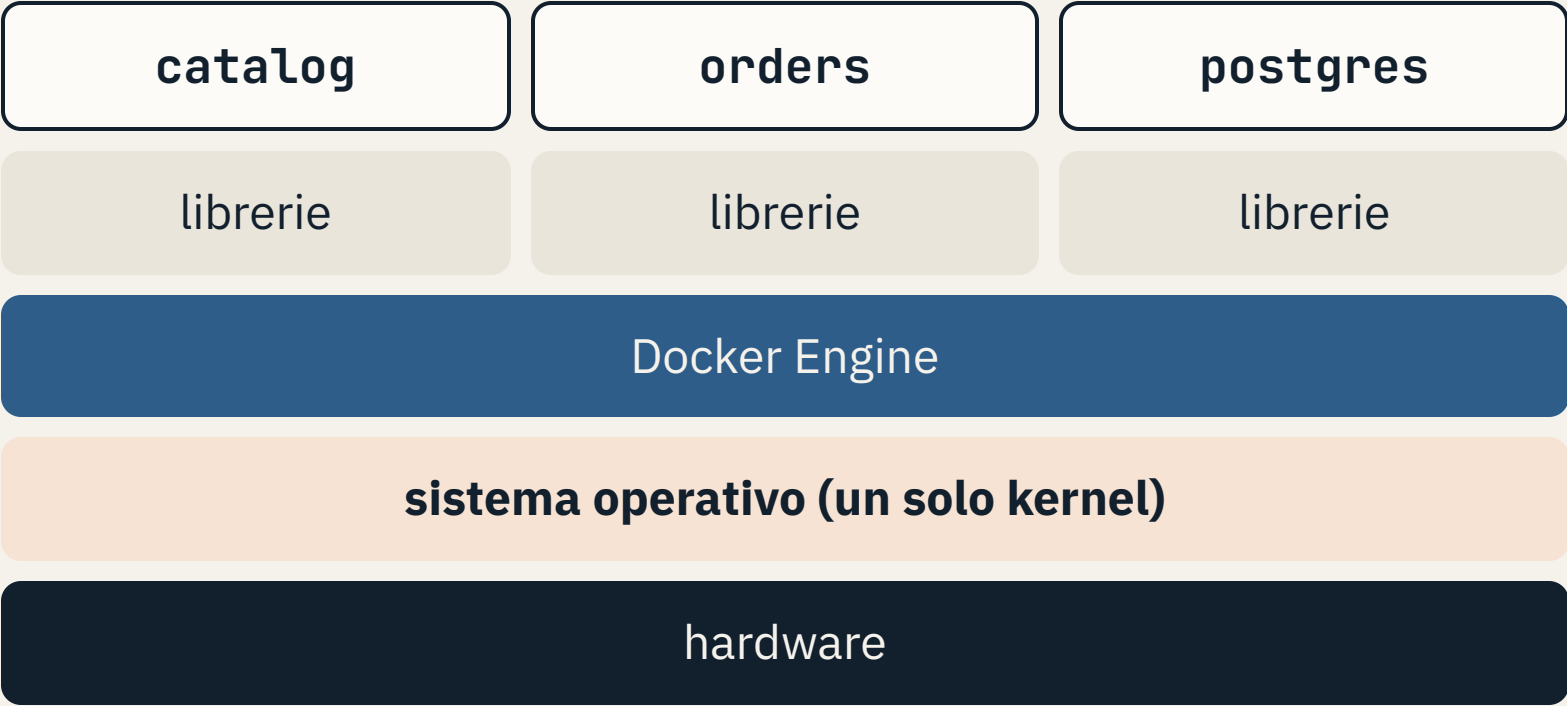
Sono funzioni del kernel Linux. Docker le rende facili da usare.

Macchina virtuale o container?

Macchine virtuali



Container



	Macchina virtuale	Container
Avvio	Decine di secondi o minuti	Circa un secondo
Dimensione tipica	Gigabyte	Decine o centinaia di megabyte
Isolamento	Molto forte: kernel separati	Buono: kernel condiviso

Immagine e container

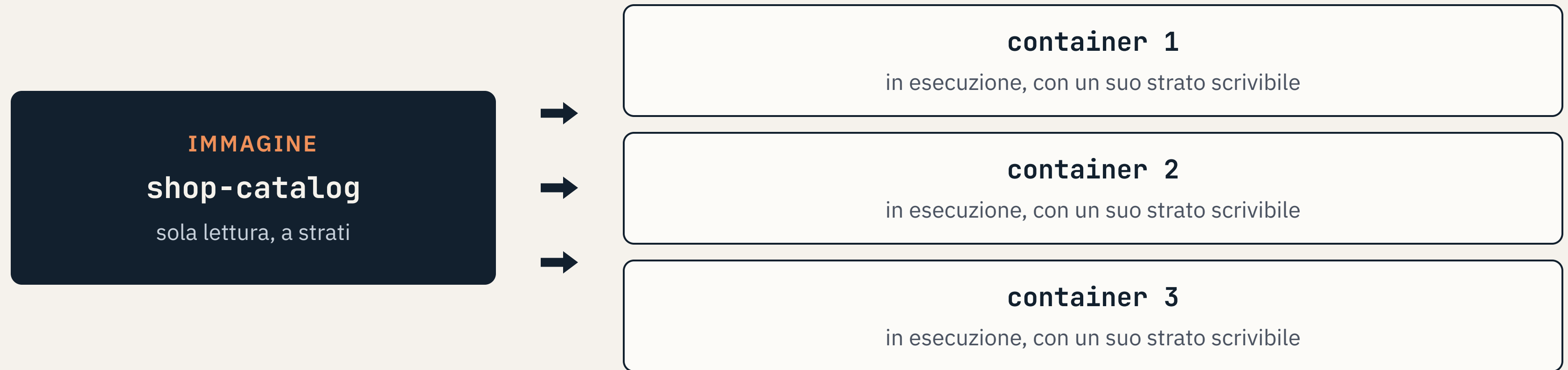


Immagine: lo stampo. Contiene file e istruzioni di avvio, non cambia mai.

Container: un'istanza in esecuzione. Da una sola immagine se ne avviano quanti se ne vuole.

Da dove arrivano le immagini

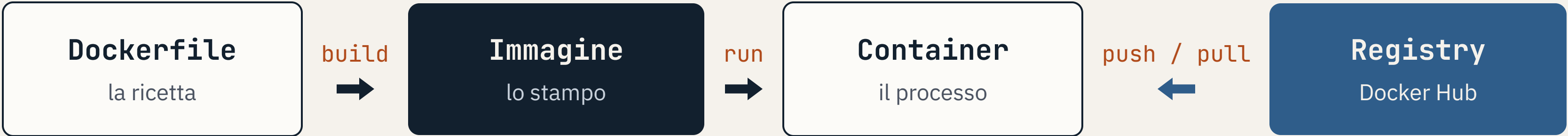
Le immagini si scaricano da un **registry**: il più usato è Docker Hub. Il nome completo si legge come un URL.

```
docker.io/library/postgres:16
```

Parte	Nell'esempio	Significato
Registry	docker.io	Dove è salvata: si può omettere per Docker Hub
Namespace	library	Chi la pubblica: library sono le immagini ufficiali
Nome	postgres	Il programma
Tag	16	La versione. Se manca vale latest

latest non vuol dire «l'ultima»: è solo un nome. Usa sempre un tag preciso, come **postgres:16** o **node:22-slim**.

Il ciclo di vita di un'immagine



Comando	A cosa serve
<code>docker build -t nome .</code>	Costruisce un'immagine dal Dockerfile della cartella
<code>docker run -p 8080:80 nome</code>	Avvia un container da un'immagine
<code>docker ps</code>	Elenca i container accesi (con -a anche quelli fermi)
<code>docker logs</code> · <code>docker exec</code>	Legge i log · lancia un comando dentro il container
<code>docker stop</code> · <code>docker rm</code>	Ferma · cancella un container

Cosa succede quando scrivi docker



Il comando **docker** non esegue i container: chiede al **demone** di farlo.

Su Mac e Windows il demone gira in una VM Linux che Docker Desktop avvia per te.



Errore **Cannot connect to the Docker daemon?** Il client funziona, ma Docker Desktop è spento: aprilo e aspetta che sia avviato.

Il Dockerfile

Istruzione	Cosa fa
FROM	Immagine di partenza
WORKDIR	Cartella di lavoro dentro l'immagine
COPY	Copia file dal tuo PC all'immagine
RUN	Esegue un comando durante la build
ENV	Imposta una variabile d'ambiente
USER	Con quale utente gira il processo
EXPOSE	Documenta la porta usata
CMD	Il comando che parte con il container

```
FROM node:22-slim
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build
CMD ["node", "dist/server.js"]
```

Si legge dall'alto in basso, come una ricetta.

Ogni istruzione è uno strato

strato scrivibile del container

sparisce con docker rm

RUN npm run build

dist/

COPY . .

il codice

RUN npm ci

node_modules, il più pesante

COPY package*.json

pochi KB

FROM node:22-slim

Debian + Node, condiviso

Gli strati sono **in sola lettura** e si impilano: l'immagine è la loro somma.

Due immagini con lo stesso FROM **condividono** quello strato: sul disco c'è una volta sola.

Il container aggiunge in cima uno strato **scrivibile**, che si perde quando il container viene cancellato.

```
docker history shop-catalog
```

La cache: l'ordine conta

Se cambia l'input di un'istruzione, Docker rifà **quella e tutte le successive**. Le precedenti escono dalla cache.

Ordine sbagliato

```
COPY . .  
RUN npm ci  
RUN npm run build
```

Modifichi una riga di **src**: COPY cambia, quindi **npm ci** riparte da zero. Ogni build costa minuti.

Ordine giusto

```
COPY package.json package-lock.json ./  
RUN npm ci # CACHED  
COPY src ./src  
RUN npm run build
```

Modifichi **src**: npm ci resta in cache. Si reinstalla solo quando cambiano le dipendenze.

Regola: prima quello che cambia di rado, poi quello che cambia spesso.

Multi-stage build

Per **compilare** servono TypeScript e le dipendenze di sviluppo. Per **eseguire** basta il JavaScript.

FROM node:22-slim AS build

tutte le dipendenze · TypeScript · src

npm run build → **dist/**

COPY --from=build



FROM node:22-slim

solo dipendenze di produzione

dist/ · node dist/server.js

Più piccola: niente compilatore, niente test.

Più sicura: meno programmi dentro, meno punti attaccabili.

Niente sorgenti nell'immagine che va in produzione.

.dockerignore e build context

docker build . spedisce al demone tutta la cartella: è il **build context**. Il file **.dockerignore** dice cosa lasciare fuori.

Da escludere	Perché
node_modules	Pesante; e quello del Mac non funziona su Linux
dist	Si ricompila dentro l'immagine
.env	Contiene password: mai in un'immagine
test, *.log	Non servono per eseguire

```
# services/catalog/.dockerignore
node_modules
dist
test
*.log
.env
```

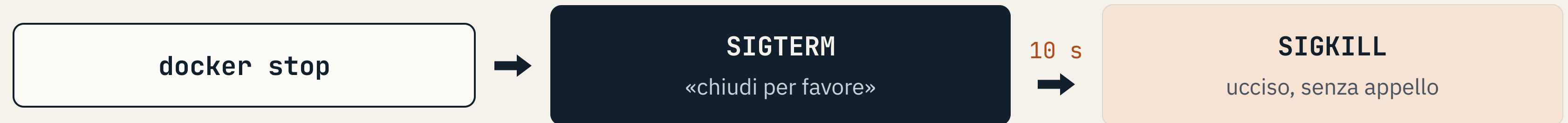


Alcuni pacchetti npm contengono programmi compilati per un sistema operativo preciso: il node_modules del Mac copiato in un container Linux può rompersi in modi strani.

Buone pratiche per le immagini

Pratica	Come	Perché
Tag precisi	node:22-slim, mai node:latest	Build ripetibili
Non usare root	USER node	Se il servizio viene bucato, il danno è limitato
CMD in forma JSON	CMD ["node", "dist/server.js"]	Il processo riceve i segnali
Un processo per container	catalog e postgres separati	Si avviano, scalano e aggiornano da soli
Config da fuori	Variabili d'ambiente	Una sola immagine per tutti gli ambienti
Niente segreti	Password mai nel Dockerfile	Chi ha l'immagine li legge
npm ci	Al posto di npm install	Usa esattamente il package-lock

docker stop e lo spegnimento pulito



```
// services/catalog/src/server.ts
process.on("SIGTERM", async () => {
  await app.close(); // richieste e DB
  process.exit(0);
});
```

Con **CMD ["node", ...]** Node è il processo principale e riceve il segnale.

Con **CMD node ...** (senza parentesi) parte prima una shell, che può non inoltrarlo: il container aspetta 10 secondi e viene ucciso.

DATI, RETE E CONFIGURAZIONE

Un container è usa e getta.

Tutto quello che scrive nel suo strato sparisce con **docker rm**. Va bene per un servizio come catalog. Per un database, no.

Tre modi di montare dati

Tipo	Esempio in compose.yaml	Quando si usa
Volume con nome	<code>catalog-data:/var/lib/postgresql/data</code>	Dati da conservare: database. Li gestisce Docker
Bind mount	<code>./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro</code>	Un file o una cartella del tuo PC: config, codice in sviluppo
tmpfs	<code>type: tmpfs, target: /tmp</code>	Dati temporanei, solo in memoria

La forma è sempre la stessa

```
sorgente:destinazione[:ro]
```

A sinistra il tuo PC o il nome del volume, a destra il percorso dentro il container.

Comandi utili

```
docker volume ls
docker compose down # tiene i volumi
docker compose down -v # li cancella
```

Porte: dentro e fuori



porta del PC : **porta del container**

Senza **-p** il servizio è acceso ma non raggiungibile dal tuo PC.

Dentro il container il server deve ascoltare su **0.0.0.0**, non su 127.0.0.1: per questo server.ts usa HOST=0.0.0.0.

Reti: i container si chiamano per nome

RETE SHOP_DEFAULT, CREATA DA COMPOSE

catalog

catalog-db:5432



catalog-db

orders

Il nome del servizio è un indirizzo

Compose ha un DNS interno: **catalog-db** diventa l'IP di quel container. Niente IP scritti a mano.

La trappola di localhost

Dentro un container **localhost** è il container stesso. Da catalog, localhost:5432 non è il database.

Configurazione con variabili d'ambiente

The Twelve-Factor App, fattore III: la configurazione sta nell'ambiente, non nel codice. Una sola immagine, tanti ambienti.

Variabile	Nel nostro catalog
PORT	Porta di ascolto, di default 3001
HOST	Interfaccia, di default 0.0.0.0
DATABASE_URL	Se c'è usa PostgreSQL, se no la memoria
NODE_ENV	production nell'immagine

```
// src/server.ts
const port = Number(process.env.PORT ?? 3001);

# dal terminale
docker run -e PORT=4000 \
  -p 4000:4000 shop-catalog
```

 Le password vanno in un file **.env** che Compose legge da solo e che **non** va nel repo: nel repo c'è solo **.env.example**.

Da tanti docker run a un file

```
docker network create shop
docker volume create catalog-data
docker run -d --name catalog-db \
  --network shop \
  -e POSTGRES_USER=shop \
  -e POSTGRES_PASSWORD=shop \
  -e POSTGRES_DB=catalog \
  -v catalog-data:/var/lib/postgresql/data \
  postgres:16
docker build -t shop-catalog ./services/catalog
docker run -d --name catalog --network shop \
  -e DATABASE_URL=postgres://... \
  -p 3001:3001 shop-catalog
```



```
docker compose up --build
```

Tutto descritto in **compose.yaml**: si legge, si mette nel repo, è uguale per tutti.

E con **down** si spegne tutto insieme.

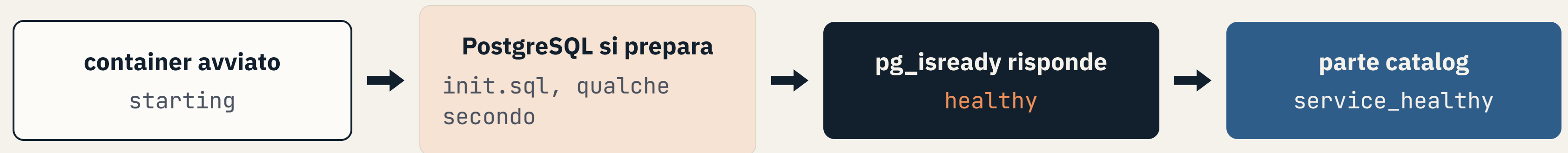
Anatomia di compose.yaml

```
services:
  catalog-db:
    image: postgres:16
    environment:
      POSTGRES_DB: catalog
    volumes:
      - catalog-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready ..."]
  catalog:
    build: ./services/catalog
    environment:
      DATABASE_URL: postgres:// ...@catalog-db ...
    ports:
      - "3001:3001"
    depends_on:
      catalog-db:
        condition: service_healthy
  volumes:
    catalog-data:
```

Chiave	Significato
image	Usa un'immagine pronta
build	Costruisci dal Dockerfile di questa cartella
environment	Variabili d'ambiente
ports	PC : container
volumes	Volumi e bind mount
depends_on	Chi deve partire prima
healthcheck	Come capire se è pronto

Partire nell'ordine giusto

depends_on da solo aspetta che il container sia **avviato**, non che il database sia **pronto**.



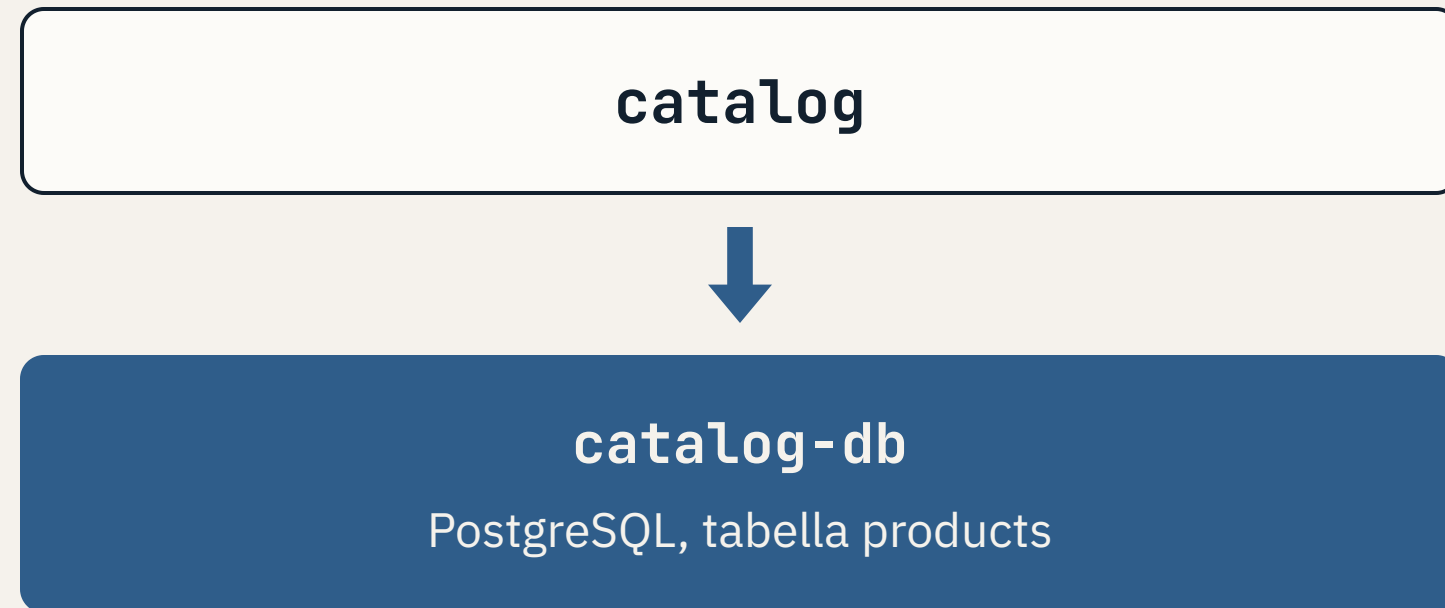
```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U shop -d catalog"]
  interval: 2s
  retries: 15
```

Docker esegue il comando ogni **2 secondi**, dentro il container. Quando riesce, il servizio diventa **healthy** e chi lo aspetta può partire.

I comandi di Compose

Comando	A cosa serve
<code>docker compose up --build</code>	Costruisce le immagini e avvia tutto, con i log a schermo
<code>docker compose up -d</code>	Avvia in background (detached)
<code>docker compose ps</code>	Stato dei servizi, con healthy / unhealthy
<code>docker compose logs -f catalog</code>	Segue i log di un servizio
<code>docker compose exec catalog-db psql ...</code>	Lancia un comando in un servizio acceso
<code>docker compose stop</code>	Ferma i container, senza cancellarli
<code>docker compose down</code>	Ferma e cancella container e rete, tiene i volumi
<code>docker compose down -v</code>	Cancella anche i volumi: il database riparte da zero
<code>docker compose config</code>	Controlla compose.yaml e lo mostra risolto

Ogni servizio, il suo database



Regola: **nessun servizio legge le tabelle di un altro**. Se a orders serve un prezzo, lo chiede a catalog.

Verifichiamo: immagini e container

1 Differenza tra immagine e container?

2 Quando si esegue RUN, e quando CMD?

3 Perché si copia package.json prima di src?

4 Cosa resta nell'immagine finale di una multi-stage build?

Verifichiamo: dati, rete e Compose

1 Perché PostgreSQL ha bisogno di un volume?

2 Da catalog, perché catalog-db e non localhost?

3 Cosa significa -p 5433:5432?

4 Che differenza c'è tra down e down -v?

Glossario

Termine	Significato
Immagine	Pacchetto in sola lettura, a strati, da cui si avviano i container.
Container	Processo isolato avviato da un'immagine.
Layer	Uno strato dell'immagine, prodotto da un'istruzione del Dockerfile.
Registry	Archivio di immagini, come Docker Hub.
Build context	La cartella inviata al demone durante docker build.
Volume	Spazio per i dati gestito da Docker, che sopravvive ai container.
Bind mount	Un file o una cartella del PC montati nel container.
Healthcheck	Comando che Docker ripete per sapere se un servizio è pronto.

Laboratorio

20'

1 · Primi passi

hello-world, nginx, logs,
exec: i comandi base.

35'

2 · Dockerfile

catalog in un'immagine
multi-stage.

40'

3 · Compose

catalog + PostgreSQL,
volume e healthcheck.

25'

4 · orders

Il secondo servizio, nel
suo container.

15'

5 · Debug

Breakpoint in VS Code
dentro un container.

```
cd microservizi/lezione-2/start → docker compose up --build
```

Il repo della lezione

github.com/mg-i-roc-maz/corsi

```
cd corsi
git pull
cd microservizi/lezione-2/start
code .
```

Non hai ancora il repo? Prima clonalo:

```
git clone https://github.com/mg-i-roc-maz/corsi
```

microservizi/lezione-2/

Cartella	Contenuto
slides.pdf	Queste slide, in PDF
start/	Si lavora qui
solution/	La soluzione, per riallinearsi
extra/	Esercizi extra e soluzioni

Cosa c'è in start

services/ catalog/ src/store/ db/init.sql Dockerfile .dockerignore orders/ Dockerfile compose.yaml compose.debug.yaml http/	memoria o PostgreSQL tabella e prodotti TODO · Parte 2 TODO · Parte 2 nuovo servizio, porta 3002 TODO · Parte 4 TODO · Parti 3 e 4 per la Parte 5 catalog.http, orders.http
---	---

Il codice TypeScript è **già pronto**: oggi si scrivono solo i file di Docker.

I test girano ancora senza Docker:

```
cd services/catalog  
npm install  
npm test # pass 10, skipped 1
```

Lo store è diventato un'interfaccia

```
// src/store/types.ts
export interface ProductStore {
  list(query?: string): Promise<Product[]>;
  get(id: string): Promise<Product | undefined>;
  create(input: ProductInput): Promise<Product>;
  ...
}

// src/store/index.ts
export function createStoreFromEnv(env = process.env) {
  return env.DATABASE_URL
    ? createPostgresStore(env.DATABASE_URL)
    : createMemoryStore();
}
```

Le rotte conoscono solo **ProductStore**: non sanno se sotto c'è una Map o PostgreSQL.

Un database risponde in modo **asincrono**: ogni metodo restituisce una Promise, e nelle rotte compare **await**.

```
const product =
  await store.get(request.params.id);
```

Parlare con PostgreSQL

```
// src/store/postgres.ts
const pool = new pg.Pool({ connectionString });

async get(id) {
  const { rows } = await pool.query(
    `SELECT id, name, price_cents AS "priceCents" ...
     FROM products WHERE id = $1`,
    [id],
  );
  return rows[0];
}
```

Parametri, non concatenazione

Il valore arriva separato, come **\$1**: il database non lo esegue mai come SQL. Così si evita la **SQL injection**.

Pool

Tiene aperte alcune connessioni e le riutilizza tra le richieste.

Dove lancio i comandi?

Terminale del computer

Terminale su Mac, PowerShell su Windows. Qui vanno **tutti i comandi docker**.

```
cd corsi/microservizi/lezione-2/start  
docker version
```

Deve mostrare sia **Client** sia **Server**.

VS Code, anche nel Dev Container

Per leggere e scrivere il codice e lanciare **npm test**, come nella lezione 1.

Il Dev Container **non ha Docker dentro**: un comando docker lì risponde command not found.



In alternativa apri lezione-2/start in VS Code **senza** Dev Container (Cmd+Shift+P → Dev Containers: Reopen Folder Locally): il terminale integrato diventa quello del computer.

1.1 · Il primo container

```
docker run hello-world
```

```
docker run -d --name web -p 8080:80 nginx
```

```
docker ps
```

Apri **http://localhost:8080**: risponde nginx, un server web che non hai installato.

Opzione	Significato
-d	In background (detached)
--name web	Un nome comodo, al posto dell'ID
-p 8080:80	Porta 8080 del PC → 80 del container
nginx	L'immagine: scaricata se manca

Prima volta: compare **Unable to find image ... Pulling**. È normale.

1.2 · Guardare dentro

```
docker logs web
docker exec -it web sh
# ora sei DENTRO il container
cat /etc/os-release
ls /usr/share/nginx/html
exit
docker stop web
docker rm web
docker images
```

Rispondi

1. Quale sistema operativo c'è nel container? E sul tuo PC?
2. Dopo docker stop ricarichi la pagina: cosa succede?
3. Dopo docker rm, docker ps -a lo mostra ancora?
4. Quanto pesa l'immagine nginx?

2.1 · Il .dockerignore di catalog

Apri **services/catalog/.dockerignore** e scrivi, una per riga, le cose da **non** mandare a Docker:

- le dipendenze installate sul PC
- il JavaScript compilato
- i test e i log
- i file con le password



Ti ricordi perché il node_modules del Mac non deve finire in un container Linux?

La sintassi è quella del .gitignore: un nome per riga, # per i commenti, * come jolly.

2.2 · TODO 1–2: la fase di build

In **services/catalog/Dockerfile**, sotto FROM node:22-slim AS build:

1. copia **solo** package.json e package-lock.json in ./
2. installa con **npm ci**
3. copia tsconfig.json, tsconfig.build.json e src
4. compila con **npm run build**

Promemoria

COPY sorgente destinazione

RUN comando

Il comando build è nuovo: **tsc -p tsconfig.build.json**
trasforma src/*.ts in dist/*.js.

2.3 · TODO 3–5: la fase di runtime

Sotto il secondo FROM node:22-slim:

3. copia i file delle dipendenze e installa solo quelle di produzione: **npm ci --omit=dev**
4. prendi dist dalla prima fase: **COPY --from=build /app/dist ./dist**
5. **USER** node, **EXPOSE** 3001 e **CMD** in forma JSON

Cosa NON deve esserci

- TypeScript e tsx
- la cartella src
- i test

Lo verifichi dopo, nella 2.5.

2.4 · Build e run

```
docker build -t shop-catalog ./services/catalog  
docker run --rm -p 3001:3001 shop-catalog
```

-t dà un nome all'immagine. **--rm** cancella il container quando lo fermi.

Prova **http/catalog.http**, poi ferma con **Ctrl+C**.

Nei log deve comparire:

```
"msg":"Archivio prodotti:  
memoria"
```

Niente DATABASE_URL, quindi memoria: crea un prodotto, ferma, riavvia. C'è ancora?

2.5 · Osserva la cache e le dimensioni

1. Cambia un commento in **src/server.ts** e ricostruisci: quali passi dicono **CACHED**?
2. Aggiungi uno spazio in **package.json** e ricostruisci: cosa cambia?
3. Guarda dimensione e strati:

```
docker images shop-catalog  
docker history shop-catalog
```

Verifica

```
docker run --rm shop-catalog ls  
docker run --rm shop-catalog \  
  ls node_modules/.bin
```

Ci sono src o tsc? Non dovrebbero.

3.1 · TODO 1–2: il database

In **compose.yaml** il servizio catalog-db ha già immagine e variabili.

Aggiungi:

1. **volumes**: il volume catalog-data sui dati di PostgreSQL, e init.sql in sola lettura
2. **healthcheck** con pg_isready

```
docker compose config
```

Se il file è scritto bene, lo stampa completo. Se no, dice la riga sbagliata.

Percorsi

```
/var/lib/postgresql/data  
/docker-entrypoint-initdb.d/
```

Il primo è dove PostgreSQL salva i dati. Il secondo contiene gli script da eseguire al primo avvio.

3.2 · TODO 3: il servizio catalog

Chiave	Valore
build	./services/catalog
environment	PORT: 3001 DATABASE_URL: postgres://shop:...@???:5432/catalog
ports	"3001:3001"
depends_on	catalog-db, condition: service_healthy



Che host metti al posto di **????** Non localhost: dentro un container localhost è il container stesso.

La password nell'URL: **`${POSTGRES_PASSWORD:-shop}`**, la stessa del database.

3.3 · Avvia il negozio

```
docker compose up --build
```

Cerca queste righe:

```
✓ Container shop-catalog-db-1 Healthy  
catalog-db-1 | ... running /docker-entrypoint-initdb.d/init.sql  
catalog-db-1 | ... ready to accept connections  
catalog-1 | ..."msg":"Archivio prodotti: PostgreSQL"
```

Da un **secondo** terminale:

```
docker compose ps  
docker compose logs catalog  
docker compose exec catalog-db \  
    psql -U shop -d catalog \  
    -c "SELECT * FROM products;"
```

3.4 · I dati sopravvivono?

1

Crea un prodotto con **catalog.http**.

```
docker compose down  
docker compose up
```

Il prodotto c'è ancora?

2

```
docker compose down -v  
docker compose up
```

E adesso? Quanti prodotti ci sono?

3

Aggiungi un quarto prodotto in **init.sql** e fai solo down e up.

Compare? Perché?

4.1 · orders esce dal monolite

Nel monolite

- CUCITURA 1: legge prezzi e scorte
- CUCITURA 2: scala le scorte
- CUCITURA 3: «manda» l'email



Nel servizio orders

Riceve l'ordine, lo valida e lo salva con **status: "pending"**.
Non sa niente dei prodotti.

```
POST /orders → 201  
{ ..., "status": "pending" }
```

Leggi **services/orders/src/routes.ts**: il commento dice dove finiranno le cuciture.

4.2 · Dockerfile e Compose per orders

1. Scrivi **services/orders/Dockerfile** partendo da quello di catalog. Cosa cambia?
2. Aggiungi il servizio **orders** in compose.yaml: build, PORT 3002, porta 3002.
3. Riavvia e prova **http/orders.http**.

```
docker compose up --build
```

Atteso

```
docker compose ps
```

Tre servizi: catalog-db (healthy), catalog e orders, tutti running.

5 · Debug di un container

```
docker compose -f compose.yaml \  
-f compose.debug.yaml up --build
```

compose.debug.yaml aggiunge a catalog:

```
command: ["node", "--enable-source-maps",  
  "--inspect=0.0.0.0:9229", "dist/server.js"]  
ports: ["9229:9229"]
```

1. In VS Code: **Run and Debug** → **Debug catalog (Docker)**
2. Breakpoint in routes.ts, dentro GET /products/:id
3. Invia la richiesta da catalog.http

Dal Dev Container scegli la configurazione **da Dev Container**.

Checkpoint

Verifica	Esito atteso
<code>docker compose ps</code>	Tre servizi running, catalog-db healthy
<code>GET localhost:3001/products</code>	I tre prodotti, letti da PostgreSQL
<code>POST localhost:3002/orders</code>	201 con "status": "pending"
<code>docker compose down, poi up</code>	I prodotti creati sono ancora lì
<code>docker compose stop catalog</code>	Subito, e nei log: SIGTERM ricevuto, chiudo il server
<code>DATABASE_URL=... npm test</code>	In services/catalog: pass 11, anche il test su PostgreSQL

Consegna: l'output di `docker compose ps` e il `compose.yaml` completato.

Se qualcosa non va

Sintomo	Causa e rimedio
Cannot connect to the Docker daemon	Docker Desktop spento: avvialo
<code>docker: command not found</code> nel terminale di VS Code	Sei nel Dev Container: usa il terminale del computer
<code>port is already allocated</code>	Porta occupata: ferma l'altro processo o cambia la porta del PC
<code>no configuration file provided</code>	Non sei nella cartella con <code>compose.yaml</code>
<code>ECONNREFUSED ...:5432</code> nei log di catalog	Manca <code>service_healthy</code> , o l'host è localhost
<code>password authentication failed</code>	Password diversa da quella con cui è nato il volume: <code>down -v</code>
Le modifiche al codice non si vedono	Manca <code>--build</code> : l'immagine è quella vecchia

ULTIMI 10 MINUTI

Cosa si rompe se...?

...fermi catalog-db mentre
catalog è acceso?

...in DATABASE_URL scrivi
localhost invece di catalog-db?

...lanci docker compose down -v
il giorno prima della consegna?

PROSSIMA LEZIONE

Lezione 3 · Progettare e far comunicare i servizi

orders chiederà a catalog prezzi e scorte via HTTP, e Nginx diventerà l'unico ingresso del negozio.

Per la volta prossima: `docker compose up --build` funzionante in lezione-2/start.

PER CHI FINISCE PRIMA, O PER CASA

Esercizi extra

#	Esercizio	Livello	Dove
1	Un'immagine più piccola con Alpine	base	Dockerfile
2	Healthcheck anche per catalog	base	compose.yaml
3	Guardare il database con Adminer	base	compose.yaml
4	La password fuori da compose.yaml	medio	.env
5	Rompere la cache dei layer	medio	Dockerfile
6	Sviluppare dentro Docker con riavvio automatico	avanzato	compose.dev.yaml

Testi completi in [lezione-2/extra/README.md](#), soluzioni in [lezione-2/extra/soluzione](#).

Un'immagine più piccola

Crea **services/catalog/Dockerfile.alpine** con **node:22-alpine** al posto di node:22-slim.

```
docker build -f services/catalog/Dockerfile.alpine \
  -t shop-catalog:alpine ./services/catalog
docker images shop-catalog
```

Domande

1. Di quanto è più piccola?
2. E una versione senza multi-stage?
3. Alpine usa musl e non glibc: quando può dare problemi?

Healthcheck anche per catalog

Aggiungi un **healthcheck** al servizio catalog che chiami **/health**.

Nell'immagine node:22-slim **non c'è curl**. Però c'è node, che ha fetch:

```
node -e "fetch('http://localhost:3001/health')
  .then(r => process.exit(r.ok ? 0 : 1))
  .catch(() => process.exit(1))"
```

Poi

Fai partire **orders** solo quando catalog è **healthy**, e verifica con docker compose ps.

Guardare il database con Adminer

Aggiungi a compose.yaml un servizio **adminer** (immagine adminer, porta 8080) e apri **http://localhost:8080**.

Campo	Valore
Sistema	PostgreSQL
Server	???
Utente · Database	shop · catalog

Domande

1. Che server scrivi? Perché non localhost?
2. Cambia un prezzo da Adminer e rileggi il prodotto via API.

La password fuori da compose.yaml

1. Copia .env.example in **.env** e cambia la password
2. Togli il valore di riserva: **\${POSTGRES_PASSWORD:?messaggio}**
3. docker compose up: catalog si collega?
4. git status: .env compare?

Suggerimento

Quando legge POSTGRES_PASSWORD l'immagine di PostgreSQL? Sempre, o solo la prima volta?

Rompere la cache dei layer

1. Nel Dockerfile sposta **COPY src ./src** prima di **RUN npm ci**
2. Cambia un commento in src/server.ts
3. Ricostruisci e misura il tempo
4. Rimetti l'ordine giusto e ripeti

```
time docker build -t shop-catalog ./services/catalog
```

Spiega in due righe

Perché una modifica a un commento costa una reinstallazione completa delle dipendenze?

Sviluppare dentro Docker

Crea **compose.dev.yaml** che per catalog:

- costruisce solo la prima fase: **build.target: build**
- monta ./services/catalog/src su /app/src
- avvia npx tsx watch src/server.ts

```
docker compose -f compose.yaml \
  -f compose.dev.yaml up --build
```

Verifica

Cambia il messaggio di /health e salva: il servizio si riavvia senza ricostruire l'immagine.

Per approfondire

Docker Docs • *Get started*

docs.docker.com: concetti e primi passi, con esercizi

Docker Docs • *Building best practices*

Cache, multi-stage, .dockerignore, utenti

Docker Docs • *Compose file reference*

Tutte le chiavi di compose.yaml

Immagine *postgres* su Docker Hub

Variabili, volumi e script di inizializzazione

The Twelve-Factor App

12factor.net: configurazione, processi, log

nodejs/docker-node • *Best Practices*

GitHub: Node.js nei container, segnali e utenti
